

ЛАБОРАТОРНАЯ РАБОТА № 5

Проектирование кэша/сессий и реализация операций Redis

Дисциплина: «Проектирование и применение NoSQL-технологий»

Образовательная программа: «Информационные системы»

Уровень обучения: бакалавриат

Курс: 3

Продолжительность: 2 академических часа

Форма выполнения: индивидуальная

Связанная лекция: Лекция № 5. «Ключи, структуры Redis, TTL, кэширование и сессии»

Основная технология: Redis

Среда выполнения: Redis CLI / Redis Insight / Docker

Форма контроля: защита результата

1. Цель работы

Сформировать практические навыки проектирования Redis-хранилища для кэширования данных и управления пользовательскими сессиями с учётом паттернов доступа, времени жизни данных, стратегии формирования ключей и правил инвалидирования кэша, а также освоить основные операции Redis для создания, чтения, изменения и автоматического удаления временных данных.

2. Результаты обучения

После выполнения лабораторной работы студент:

1. **анализирует** требования информационной системы и определяет данные, подходящие для размещения в Redis;
2. **проектирует** пространство Redis-ключей на основе пользовательских сценариев и паттернов доступа;
3. **выбирает и применяет** подходящие структуры Redis: String, Hash, List, Set и Sorted Set;

4. **создаёт** кэшированные данные и пользовательские сессии с ограниченным временем жизни;
 5. **применяет** команды **SET**, **GET**, **HSET**, **HGET**, **HGETALL**, **EXPIRE**, **TTL**, **DEL** и другие необходимые операции;
 6. **проверяет** корректность TTL и автоматического удаления временных данных;
 7. **обосновывает** стратегию кэширования, правила инвалидирования и ограничения разработанного решения.
-

3. Необходимые знания

Перед выполнением работы необходимо повторить:

- назначение NoSQL-систем;
- модель «ключ — значение» (key-value);
- понятие ключа Redis;
- основные типы данных Redis;
- клиент-серверную архитектуру;
- понятие кэширования;
- cache hit и cache miss;
- пользовательскую сессию;
- TTL — Time To Live;
- сериализацию данных;
- идентификаторы объектов;
- основные операции CRUD;
- причины появления устаревших данных в кэше.

Студент должен понимать принцип разделения постоянного хранилища данных и временного быстрого хранилища.

4. Необходимое оборудование и программное обеспечение

4.1. Основное ПО

Рекомендуется использовать:

- Redis Open Source актуальной стабильной версии;
- Redis CLI;
- Redis Insight — по желанию;
- Docker Desktop — для запуска Redis в контейнере;
- терминал операционной системы.

Для данной лабораторной работы написание отдельного приложения на Java или Python не является обязательным.

4.2. Минимальные требования

- 64-разрядная ОС;
- 2 ГБ оперативной памяти;
- около 1 ГБ свободного дискового пространства;
- Docker Desktop либо локальная установка Redis;
- возможность работы с командной строкой.

4.3. Запуск через Docker

Для учебной локальной среды:

```
docker run --name redis-lab5 -p 127.0.0.1:6379:6379 -d redis:8
```

Проверить состояние:

```
docker ps
```

Подключиться к Redis CLI:

```
docker exec -it redis-lab5 redis-cli
```

Проверить соединение:

```
PING
```

Ожидаемый результат:

```
PONG
```

Привязка порта к `127.0.0.1` используется для того, чтобы учебный экземпляр Redis не открывался непосредственно во внешнюю сеть.

5. Правила безопасности

При выполнении лабораторной работы необходимо соблюдать следующие требования:

1. Не использовать реальные персональные данные студентов, сотрудников, клиентов и пациентов.
 2. Использовать только искусственно созданные учебные данные.
 3. Не публиковать пароли, токены, session ID и строки подключения.
 4. Не открывать Redis во внешнюю сеть без необходимости.
 5. Не применять учебные команды к рабочим базам данных.
 6. Перед `DEL`, `UNLINK` и другими операциями удаления обязательно проверить ключ.
 7. Не использовать `FLUSHALL` и `FLUSHDB` в общей или рабочей среде.
 8. Не использовать слабые или предсказуемые идентификаторы реальных пользовательских сессий.
 9. При работе через Docker ограничивать публикацию Redis-порта локальным интерфейсом.
 10. В отчёте использовать только учебные идентификаторы пользователей.
-

6. Краткие теоретические сведения

6.1. Redis

Redis — Remote Dictionary Server — высокопроизводительное хранилище структур данных, в котором доступ к данным в типичном случае осуществляется по ключу.

Простейшая логическая модель:

ключ → значение

Например:

student:1001:name → "Aruzhan"

Основное преимущество Redis для рассматриваемой задачи заключается в возможности быстро получать временные данные по известному ключу.

Redis особенно удобен для:

- кэширования;
- хранения сессий;
- счётчиков;
- временных токенов;
- очередей;
- рейтингов;
- ограничения частоты запросов.

Redis не следует автоматически рассматривать как замену основной реляционной или документной БД. В рассматриваемой лабораторной работе Redis используется преимущественно как специализированный компонент информационной системы.

7. Ключи Redis

Key — уникальное имя объекта в Redis.

При проектировании системы важно заранее определить соглашение об именовании ключей.

Рекомендуемый шаблон:

<тип>:<сущность>:<идентификатор>

Например:

```
cache:student:1001
session:a8f27c
course:CS305
cart:user:1001
```

Для вложенной логической группировки можно использовать дополнительные части:

```
cache:course:CS305:materials
```

Хорошее имя ключа должно:

- отражать назначение данных;
 - позволять избежать конфликтов;
 - использовать единый стиль;
 - содержать необходимые идентификаторы;
 - быть достаточно коротким.
-

8. Основные структуры данных Redis

8.1. String

String — простейший тип значения Redis.

Пример:

```
SET cache:course:CS305:name "NoSQL Technologies"  
GET cache:course:CS305:name
```

String удобен для:

- строк;
 - чисел;
 - сериализованных JSON-объектов;
 - токенов;
 - флагов;
 - счётчиков.
-

8.2. Hash

Hash представляет объект как набор пар:

поле → значение

Например:

```
HSET session:demo001 user_id "1001" role "student" language "ru"
```

Получение всех полей:

```
HGETALL session:demo001
```

Hash хорошо подходит для небольших объектов, свойства которых необходимо изменять независимо.

8.3. List

List представляет упорядоченную последовательность элементов.

Пример:

```
LPUSH recent:student:1001 "course:CS305"
```

```
LPUSH recent:student:1001 "course:DB201"
```

Получение:

```
LRANGE recent:student:1001 0 -1
```

List может применяться для:

- последних действий;
 - простых очередей;
 - истории последних просмотренных объектов.
-

8.4. Set

Set — неупорядоченное множество уникальных элементов.

```
SADD online:course:CS305 student:1001 student:1002
```

Проверка:

```
SMEMBERS online:course:CS305
```

Множества удобны там, где один элемент не должен повторяться.

8.5. Sorted Set

Sorted Set хранит уникальные элементы вместе с числовой оценкой — score.

Например:

```
ZADD course:CS305:activity 85 student:1001
```

```
ZADD course:CS305:activity 72 student:1002
```

Получение рейтинга:

```
ZREVRANGE course:CS305:activity 0 -1 WITHSCORES
```

9. TTL — время жизни данных

TTL — Time To Live — оставшееся время существования ключа.

Для кэша и пользовательских сессий это одна из наиболее важных возможностей Redis.

Пример:

```
SET session:temp001 "user:1001" EX 1800
```

Ключ автоматически должен перестать существовать после установленного периода.

Проверить оставшееся время:

```
TTL session:temp001
```

Также TTL можно установить отдельно:

```
EXPIRE session:temp001 1800
```

Команда:

TTL session:temp001

возвращает:

- положительное число — количество оставшихся секунд;
- **-1** — ключ существует, но срок жизни не установлен;
- **-2** — ключ отсутствует.

TTL позволяет автоматически удалять временные данные без отдельной ручной операции очистки.

10. Кэширование

Cache — промежуточное быстрое хранилище результатов, получение которых из основной системы требует больше времени или ресурсов.

Типичный сценарий:



Cache hit

Необходимые данные обнаружены в Redis.

Cache miss

Данных в Redis нет, поэтому приложение обращается к основному источнику.

После получения данных результат может быть помещён в Redis.

11. Инвалидирование кэша

Главная проблема кэширования заключается не только в сохранении данных, но и в определении момента, когда кэшированные данные больше нельзя использовать.

Рассмотрим:

Основная БД:

course CS305 → room 305

Redis:

cache:course:CS305 → room 305

Если в основной БД аудитория изменена на 410, а Redis продолжает возвращать 305, возникает устаревшее значение.

Возможные стратегии:

TTL-based invalidation

Данные автоматически удаляются после установленного времени.

Explicit invalidation

После изменения основной записи приложение удаляет соответствующий кэш:

DEL cache:course:CS305

Update cache

После изменения основной записи приложение одновременно обновляет значение в Redis.

Выбор стратегии зависит от допустимого времени устаревания данных.

12. Пользовательская сессия

Сессия содержит временное состояние пользователя между запросами.

Пример:

```
session:2fd934
  user_id = 1001
  role = student
  language = ru
```

Сессия обычно должна иметь ограниченное время жизни.

Например:

```
HSET session:demo001 user_id "1001" role "student" language "ru"
EXPIRE session:demo001 1800
```

Проверка:

```
TTL session:demo001
```

Redis удобен для централизованного хранения сессий, когда информационная система содержит несколько экземпляров приложения.

13. Постановка практической задачи

Рассматривается информационная система университета.

В системе работают:

- студенты;
- преподаватели;
- сотрудники;
- администраторы.

Основная БД содержит постоянную информацию:

- пользователей;

- дисциплины;
- расписание;
- образовательные программы;
- регистрации студентов;
- учебные материалы.

Часть информации запрашивается значительно чаще, чем изменяется.

Например:

информация о дисциплине;
краткое расписание;
настройки интерфейса;
данные активной пользовательской сессии.

Предполагается следующая нагрузка:

- тысячи операций чтения;
- значительно меньше операций изменения;
- частое повторное чтение одинаковых данных;
- необходимость быстро проверять активность сессии;
- временный характер кэшированных данных.

Необходимо разработать Redis-модель, которая позволяет:

1. кэшировать часто запрашиваемые данные;
2. хранить активные пользовательские сессии;
3. автоматически удалять временные записи;
4. явно инвалидировать устаревшие данные;
5. различать типы данных по ключам.

14. Подготовка среды

Шаг 1. Запуск Redis

```
docker run --name redis-lab5 -p 127.0.0.1:6379:6379 -d redis:8
```

Ожидаемый результат: контейнер Redis запущен.

Проверка:

`docker ps`

Возможная ошибка:

`port is already allocated`

Причина: порт 6379 уже используется.

Проверить существующие контейнеры:

`docker ps -a`

Шаг 2. Подключение

`docker exec -it redis-lab5 redis-cli`

Проверка:

PING

Результат:

PONG

Шаг 3. Проверка версии

`INFO server`

Найдите параметр:

`redis_version`

Шаг 4. Проверка учебной среды

Создайте:

SET lab5:test "Redis works"

Проверьте:

GET lab5:test

Ожидаемый результат:

"Redis works"

Удалите тестовый ключ:

DEL lab5:test

15. Демонстрационный пример

Рассмотрим небольшой пример кэширования информации о дисциплине.

15.1. Создание кэша

HSET cache:course:CS305 name "NoSQL Technologies" teacher "A. Teacher" room "305"

Проверка:

HGETALL cache:course:CS305

Ожидается получение полей:

name
NoSQL Technologies
teacher
A. Teacher
room
305

15.2. Установка TTL

EXPIRE cache:course:CS305 600

Проверка:

TTL cache:course:CS305

Результат должен быть положительным и не превышать 600 секунд.

15.3. Чтение отдельного поля

HGET cache:course:CS305 room

Результат:

"305"

15.4. Изменение данных

Предположим, аудитория была изменена:

HSET cache:course:CS305 room "410"

Проверка:

HGET cache:course:CS305 room

Результат:

"410"

Проверьте TTL:

TTL cache:course:CS305

15.5. Инвалидирование

Если основная запись была изменена, кэш можно удалить:

DEL cache:course:CS305

Проверка:

EXISTS cache:course:CS305

Ожидаемый результат:

0

Это имитирует ситуацию, когда приложение после изменения основной БД принудительно удаляет устаревший кэш.

16. Пошаговый порядок выполнения

Этап 1. Анализ требований

Для своего индивидуального варианта определите:

1. пользователей системы;
2. часто выполняемые операции;
3. операции записи;
4. данные, которые можно кэшировать;
5. данные, которые нельзя считать допустимо устаревающими;
6. данные пользовательской сессии;
7. ожидаемую продолжительность сессии;
8. допустимое время устаревания кэша.

Составьте таблицу:

Данные	Операция	Частота	Можно кэшировать?	TTL	Причина
Карточка курса	чтение	высокая	да	600 с	редко изменяется
Оценка	чтение/запись	средняя	ограниченно	—	требуется актуальность
Сессия	чтение/запись	высокая	Redis используется как временное хранилище	1800 с	временные данные

17. Этап 2. Выбор модели

Сравните минимум два подхода.

Например:

Критерий	Основная БД	Redis
Постоянное хранение	подходит	не основная задача данного решения
Быстрое чтение по ключу	зависит от БД	подходит
TTL	зависит от СУБД	встроенный механизм
Сессии	возможно	удобно
Кэш	возможно	специализированный сценарий

Сделайте вывод:

- какие данные остаются в основной БД;
 - какие помещаются в Redis;
 - является ли Redis кэшем, session store или выполняет обе функции.
-

18. Этап 3. Проектирование ключей

Разработайте минимум 8 различных ключей.

Например:

cache:course:CS305
cache:course:DB201
session:s001
session:s002
recent:user:1001
online:course:CS305
counter:course:CS305:views
activity:course:CS305

Для каждого укажите:

Ключ	Тип Redis	Назначение	TTL
cache:course:CS305	Hash	кэш курса	600
session:s001	Hash	сессия	1800
counter:course:CS305:views	String	счётчик	без TTL/по требованиям

Обоснуйте каждую выбранную структуру.

19. Этап 4. Создание тестовых данных

Создайте не менее **15 осмысленных записей или элементов**.

Запрещено использовать бессмысленные данные:

test1
value1
abc
123

Данные должны соответствовать предметной области.

Пример:

HSET cache:course:CS305 name "NoSQL Technologies" semester "5" teacher "Teacher A"

HSET cache:course:DB201 name "Database Systems" semester "4" teacher "Teacher B"

20. Этап 5. Работа с пользовательскими сессиями

Создайте минимум три учебные сессии.

```
HSET session:s001 user_id "1001" role "student" language "ru"  
EXPIRE session:s001 1800
```

```
HSET session:s002 user_id "1002" role "student" language "kk"  
EXPIRE session:s002 1800
```

```
HSET session:s003 user_id "501" role "teacher" language "ru"  
EXPIRE session:s003 900
```

Проверьте:

```
HGETALL session:s001  
TTL session:s001
```

Объясните, почему сессия должна иметь TTL.

21. Этап 6. Операции с кэшем

Создайте не менее трёх кэшированных объектов.

Например:

```
HSET cache:course:CS305 name "NoSQL Technologies" room "305"  
EXPIRE cache:course:CS305 600
```

Выполните:

1. создание;

2. чтение;
3. проверку TTL;
4. изменение;
5. инвалидирование одного объекта.

Контроль:

EXISTS cache:course:CS305

22. Этап 7. Счётчик

Создайте счётчик просмотров:

SET counter:course:CS305:views 0

Увеличьте:

INCR counter:course:CS305:views

INCR counter:course:CS305:views

INCR counter:course:CS305:views

Проверка:

GET counter:course:CS305:views

Объясните, почему для такого сценария подходит Redis String.

23. Этап 8. Множество активных пользователей

SADD online:course:CS305 student:1001 student:1002 teacher:501

Получите список:

SMEMBERS online:course:CS305

Проверьте количество:

SCARD online:course:CS305

Удалите пользователя:

SREM online:course:CS305 student:1002

24. Этап 9. Последние просмотренные объекты

LPUSH recent:user:1001 "course:CS305"

LPUSH recent:user:1001 "course:DB201"

LPUSH recent:user:1001 "course:WEB301"

Получите последние элементы:

LRANGE recent:user:1001 0 4

Ограничьте список:

LTRIM recent:user:1001 0 4

Объясните, почему ограничение размера списка важно.

25. Этап 10. Проверка TTL

Создайте короткоживущий тестовый ключ:

SET cache:tfl-demo "temporary" EX 20

Сразу выполните:

```
TTL cache:ttl-demo
```

Через несколько секунд повторите:

```
TTL cache:ttl-demo
```

После истечения TTL:

```
GET cache:ttl-demo
```

Ожидается отсутствие значения.

26. Этап 11. Инвалидирование

Смоделируйте изменение данных в основной БД.

Исходный кэш:

```
HSET cache:course:CS305 room "305"  
EXPIRE cache:course:CS305 600
```

Предположим:

Основная БД:
room = 410

Удалите устаревший кэш:

```
DEL cache:course:CS305
```

Проверка:

```
EXISTS cache:course:CS305
```

Ответьте:

1. Что произойдёт при следующем запросе приложения?
 2. Откуда приложение должно получить актуальные данные?
 3. Нужно ли снова сохранить их в Redis?
 4. Какой TTL следует использовать?
 5. Что произойдёт, если инвалидирование не выполнить?
-

27. Основное индивидуальное задание

Для назначенного преподавателем варианта необходимо:

1. проанализировать предметную область;
 2. определить минимум пять паттернов доступа;
 3. определить данные для кэширования;
 4. определить данные пользовательской сессии;
 5. спроектировать соглашение об именовании ключей;
 6. использовать минимум три подходящих типа данных Redis;
 7. создать не менее 15 осмысленных записей/элементов;
 8. реализовать минимум пять операций чтения;
 9. выполнить изменение данных;
 10. выполнить безопасное удаление;
 11. использовать TTL;
 12. реализовать одно аналитическое действие;
 13. продемонстрировать инвалидирование;
 14. проверить результаты;
 15. обосновать выбранную Redis-модель.
-

28. Индивидуальные варианты

Вариант 1. Информационная система университета

Сущности: студент, преподаватель, дисциплина, сессия, расписание.

Обязательные поля сессии:

user_id

role
language
last_section

Спроектировать:

session:<id>
cache:course:<id>
cache:schedule:<group>
recent:student:<id>
course:<id>:online

Минимум пять запросов:

1. получить сессию студента;
2. получить кэш дисциплины;
3. проверить TTL сессии;
4. получить последние просмотренные дисциплины;
5. получить активных пользователей дисциплины.

Изменение: изменить язык интерфейса в сессии.

Удаление: завершить выбранную сессию.

Аналитика: определить количество активных пользователей дисциплины.

Повышенная сложность: разработать стратегию инвалидирования расписания после изменения аудитории.

Обоснование: объяснить, почему Redis используется только для выбранных данных.

Вариант 2. Интернет-магазин

Сущности: пользователь, товар, корзина, категория, сессия.

Обязательные поля:

product_id
name
price
category
user_id

Ключи:

session:<id>
cache:product:<id>
cart:user:<id>
recent:user:<id>
product:<id>:views

Запросы:

1. получить товар из кэша;
2. получить корзину;
3. проверить сессию;
4. получить последние просмотренные товары;
5. получить счётчик просмотров.

Изменение: изменить количество товара в корзине.

Удаление: удалить товар из корзины.

Аналитика: определить наиболее просматриваемый товар среди тестовых данных.

Повышенная сложность: предложить стратегию инвалидирования цены.

Обоснование: объяснить, почему цена требует более строгой актуальности, чем описание товара.

Вариант 3. Библиотечная система

Сущности: читатель, книга, автор, категория, сессия.

Поля:

book_id
title
author
category
user_id

Ключи:

cache:book:<id>
session:<id>
recent:reader:<id>
book:<id>:views
online:library

Запросы:

1. получить карточку книги;
2. получить сессию;
3. проверить TTL;
4. получить последние просмотренные книги;
5. получить количество активных читателей.

Изменение: изменить настройки языка пользователя.

Удаление: инвалидировать кэш книги.

Аналитика: подсчитать просмотры выбранной книги.

Повышенная сложность: определить, какие данные о доступности экземпляров допустимо кэшировать.

Вариант 4. Медицинская информационная система

Используются только **искусственные учебные данные**.

Сущности: пользователь, врач, подразделение, расписание, сессия.

Поля:

user_id
role
department
language
schedule_id

Ключи:

session:<id>
cache:doctor:<id>
cache:schedule:<id>

recent:user:<id>
department:<id>:online

Запросы:

1. получить сессию;
2. получить кэш общедоступного расписания;
3. проверить TTL;
4. получить последние просмотренные разделы;
5. получить число активных пользователей подразделения.

Изменение: изменить язык сессии.

Удаление: удалить завершённую сессию.

Аналитика: определить число активных сессий в тестовом подразделении.

Повышенная сложность: определить, какие медицинские данные нельзя помещать в такой учебный кэш без дополнительных требований безопасности.

Вариант 5. Служба доставки

Сущности: клиент, заказ, курьер, зона доставки, сессия.

Поля:

user_id
order_id
courier_id
zone
status

Ключи:

session:<id>
cache:delivery-zone:<id>
cache:courier:<id>
recent:customer:<id>
courier:<id>:activity

Запросы:

1. получить сессию клиента;
2. получить данные зоны;
3. получить кэш курьера;
4. получить последние просмотренные заказы;
5. получить показатель активности.

Изменение: изменить выбранную зону доставки.

Удаление: инвалидировать кэш курьера.

Аналитика: подсчитать обращения к выбранной зоне.

Повышенная сложность: выбрать TTL для данных, которые изменяются с разной частотой.

Вариант 6. Система бронирования

Сущности: пользователь, объект, номер/ресурс, бронирование, сессия.

Поля:

user_id
resource_id
name
location
language

Ключи:

session:<id>
cache:resource:<id>
recent:user:<id>
resource:<id>:views
online:booking

Запросы:

1. получить кэш объекта;
2. получить сессию;
3. проверить TTL;
4. получить последние просмотренные объекты;

5. получить число активных пользователей.

Изменение: изменить настройку сессии.

Удаление: инвалидировать карточку объекта.

Аналитика: определить число просмотров объектов.

Повышенная сложность: объяснить, почему фактическую доступность ресурса нельзя бездумно кэшировать на длительный срок.

Вариант 7. Социальная платформа

Сущности: пользователь, публикация, профиль, сессия, активность.

Поля:

user_id
username
language
post_id

Ключи:

session:<id>
cache:profile:<id>
recent:user:<id>
post:<id>:views
online:users

Запросы:

1. получить профиль;
2. получить сессию;
3. получить последние просмотренные публикации;
4. проверить TTL;
5. получить число пользователей онлайн.

Изменение: изменить настройку языка.

Удаление: удалить сессию.

Аналитика: определить количество просмотров публикации.

Повышенная сложность: разработать правило инвалидирования кэша профиля после его редактирования.

Вариант 8. Складской учёт

Сущности: сотрудник, товар, склад, категория, сессия.

Поля:

product_id
name
category
warehouse_id
user_id

Ключи:

session:<id>
cache:product:<id>
cache:warehouse:<id>
recent:employee:<id>
product:<id>:views

Запросы:

1. получить карточку товара;
2. получить информацию о складе;
3. получить сессию;
4. проверить TTL;
5. получить последние просмотренные товары.

Изменение: изменить параметр сессии.

Удаление: инвалидировать кэш товара.

Аналитика: подсчитать обращения к карточкам товаров.

Повышенная сложность: объяснить, почему фактический остаток товара требует осторожной стратегии кэширования.

Вариант 9. Платформа онлайн-обучения

Сущности: студент, курс, урок, преподаватель, сессия.

Поля:

user_id
course_id
lesson_id
language
role

Ключи:

session:<id>
cache:course:<id>
cache:lesson:<id>
recent:student:<id>
course:<id>:online

Запросы:

1. получить кэш курса;
2. получить кэш урока;
3. получить сессию;
4. получить последние уроки;
5. получить число пользователей курса онлайн.

Изменение: изменить последний открытый урок.

Удаление: удалить сессию.

Аналитика: определить количество активных пользователей курса.

Повышенная сложность: спроектировать различные TTL для описания курса и текущего состояния пользователя.

Вариант 10. Система мониторинга устройств

Сущности: устройство, пользователь, состояние устройства, группа, сессия.

Поля:

device_id
name
status
group
updated_at

Ключи:

session:<id>
cache:device:<id>
recent:user:<id>
device:<id>:requests
online:devices

Запросы:

1. получить состояние устройства;
2. получить сессию;
3. проверить TTL состояния;
4. получить список недавно просмотренных устройств;
5. получить активные устройства.

Изменение: изменить тестовое состояние устройства.

Удаление: инвалидировать кэш устройства.

Аналитика: определить число активных устройств.

Повышенная сложность: обосновать короткий TTL для быстро изменяющегося состояния.

29. Задания по уровням сложности

Базовый уровень

Необходимо:

- запустить Redis;
- проверить соединение;
- разработать ключи;
- создать минимум 15 элементов;
- использовать String и Hash;
- выполнить чтение;
- выполнить изменение;
- выполнить удаление;
- установить TTL;
- проверить TTL.

Выполнение только базового уровня позволяет претендовать максимум на **69 баллов**.

Средний уровень

Дополнительно необходимо:

- использовать минимум три структуры Redis;
- реализовать сессию;
- реализовать кэш;
- использовать Set или List;
- реализовать счётчик;
- выполнить минимум пять запросов;
- продемонстрировать cache invalidation;
- обосновать TTL;
- проанализировать результаты.

Полное выполнение базового и среднего уровней позволяет претендовать на **70–89 баллов**.

Повышенный уровень

Необходимо дополнительно:

- сравнить две стратегии инвалидирования;
- обосновать TTL для разных категорий данных;
- выявить риск устаревших данных;
- предложить улучшенную архитектуру;
- продемонстрировать поведение ключа до и после истечения TTL.

Полное выполнение всех обязательных требований позволяет претендовать на **90–100 баллов**.

30. Ожидаемые результаты

После выполнения работы должны существовать:

- спроектированное пространство ключей;
- минимум 15 осмысленных элементов;
- минимум три типа структур Redis;
- кэшированные данные;
- минимум три тестовые сессии;
- ключи с TTL;
- счётчик или другая аналитическая структура;
- операция изменения;
- операция удаления;
- демонстрация инвалидирования.

Студент должен уметь показать преподавателю:

```
TYPE <key>
GET <key>
HGETALL <key>
TTL <key>
EXISTS <key>
```

в зависимости от используемого типа.

31. Проверка результата

Для каждого ключевого объекта выполните:

```
TYPE <key>
```

Проверьте наличие:

```
EXISTS <key>
```

Для Hash:

HGETALL <key>

Для String:

GET <key>

Для List:

LRANGE <key> 0 -1

Для Set:

SMEMBERS <key>

Для TTL:

TTL <key>

Успешным считается результат, если:

- структура соответствует проекту;
- ключ имеет правильный тип;
- данные читаются;
- изменение отражается в результате;
- удалённый объект отсутствует;
- TTL установлен там, где это требуется;
- временный объект исчезает после окончания TTL.

32. Анализ разработанного решения

В отчёте письменно ответьте:

1. Почему для задачи выбран Redis?
2. Какие данные помещены в Redis?

3. Какие данные должны оставаться в основной БД?
 4. Почему выбраны именно эти структуры Redis?
 5. Как формируются ключи?
 6. Почему выбран такой TTL?
 7. Какие данные могут устареть?
 8. Какая стратегия инвалидирования используется?
 9. Что произойдёт при cache miss?
 10. Как изменится архитектура при росте количества пользователей?
 11. Какие данные нельзя кэшировать надолго?
 12. Что произойдёт при неправильном выборе TTL?
-

33. Типичные ошибки

Ошибка	Причина	Последствие	Исправление
Redis используется как копия всей основной БД	отсутствует анализ требований	усложнение архитектуры	определить конкретную роль Redis
Непонятные ключи <code>key1, data2</code>	нет соглашения об именовании	сложность сопровождения	использовать namespace
Сессия без TTL	забыто время жизни	накопление старых сессий	устанавливать TTL
Слишком большой TTL	недооценена изменчивость	устаревшие данные	выбирать TTL по требованиям
Слишком маленький TTL	чрезмерная осторожность	частые cache miss	анализировать частоту изменений
<code>SET</code> применяется к существующему ключу без анализа TTL	не учтена семантика операции	прежний TTL может быть потерян	после изменения проверять <code>TTL</code> либо использовать подходящую команду
<code>DEL</code> выполняется без проверки	неправильный ключ	потеря данных	сначала <code>GET</code> , <code>TYPE</code> , <code>EXISTS</code>
Один огромный список без ограничения	отсутствует контроль роста	расход памяти	применять <code>LTRIM</code>

Реальные session ID в отчёте	нарушение безопасности	раскрытие данных	использовать учебные значения
Кэш не инвалидируется	изменение основной БД не связано с Redis	пользователь получает устаревшие данные	разработать стратегию инвалидирования

34. Контрольные вопросы

Ключевые понятия

1. Что представляет собой модель key-value и как она реализуется в Redis?
2. Что такое TTL и зачем он необходим для кэша и пользовательских сессий?
3. Чем cache hit отличается от cache miss?

Команды

4. Чем отличаются команды `GET` и `HGET`?
5. Как с помощью `EXPIRE` и `TTL` установить и проверить время жизни ключа?

Моделирование

6. Почему ключ `session:<id>` предпочтительнее абстрактного `data1`?
7. В каком случае данные пользователя удобнее хранить в Hash, а не в нескольких независимых String-ключях?

Анализ результатов

8. Что означает результат `TTL = -1` и почему он может быть проблемой для сессии?
9. Что произойдёт, если информация в основной БД изменится, а соответствующий кэш Redis останется прежним?

Архитектура

10. Почему Redis в рассматриваемой архитектуре не должен автоматически заменять основную БД информационной системы?
-

35. Требования к отчёту

Отчёт должен содержать:

1. титульный лист;
2. номер и точное название лабораторной работы;
3. цель;
4. номер индивидуального варианта;
5. описание предметной области;
6. пользователей системы;
7. анализ требований;
8. таблицу паттернов доступа;
9. обоснование применения Redis;
10. схему ключей;
11. таблицу типов данных и TTL;
12. команды Redis;
13. тестовые данные;
14. результаты выполнения запросов;
15. демонстрацию сессии;
16. демонстрацию кэширования;
17. демонстрацию TTL;
18. демонстрацию инвалидирования;
19. скриншоты;
20. анализ решения;
21. ответы на контрольные вопросы;
22. вывод.

Каждый скриншот должен иметь:

Рисунок N — Название результата

и пояснение в 1–3 предложениях.

Скриншот без пояснения не считается полноценным подтверждением выполнения задания.

36. Требования к выводу

В выводе необходимо указать:

- что было спроектировано;
 - какие структуры Redis использовались;
 - как организованы ключи;
 - где применялся TTL;
 - как реализована сессия;
 - как реализован кэш;
 - каким способом решается проблема устаревших данных;
 - какие ограничения имеет разработанная модель;
 - какие практические навыки были получены.
-

37. Критерии оценивания

Критерий	Баллы
Анализ требований	10
Обоснование выбора модели и роли Redis	10
Проектирование ключей и структур	15
Корректность тестовых данных	10
Выполнение основных операций	20
Кэширование, TTL и сложные операции	15
Анализ результатов	10
Оформление отчёта	5
Ответы при защите	5
Итого	100

Интерпретация критериев

Анализ требований — 10 баллов

Полное выполнение: определены пользователи, операции, частота доступа, требования к актуальности и TTL.

Частичное: требования перечислены без анализа.

Невыполнение: Redis-модель создана без предварительного анализа.

Обоснование модели — 10 баллов

Полное: студент объясняет роль Redis и границу между Redis и основной БД.

Частичное: дано общее объяснение.

Невыполнение: выбор не обоснован.

Проектирование структуры — 15 баллов

Полное: понятные ключи, подходящие структуры и TTL.

Частичное: имеются отдельные ошибки моделирования.

Невыполнение: ключи и структуры случайны.

Тестовые данные — 10 баллов

Полное: минимум 15 осмысленных элементов.

Частичное: недостаточный объём или однообразные данные.

Невыполнение: отсутствуют пригодные тестовые данные.

Основные операции — 20 баллов

Полное: создание, чтение, изменение, удаление и проверка выполнены корректно.

Частичное: отдельные операции отсутствуют.

Невыполнение: команды не воспроизводятся.

Кэширование и TTL — 15 баллов

Полное: реализованы кэш, сессия, TTL и инвалидирование.

Частичное: реализованы не все механизмы.

Невыполнение: TTL и стратегия кэша отсутствуют.

Анализ результатов — 10 баллов

Полное: студент анализирует преимущества, риски и ограничения.

Частичное: вывод носит описательный характер.

Невыполнение: анализ отсутствует.

Оформление — 5 баллов

Полное: отчёт структурирован, команды и скриншоты прокомментированы.

Защита — 5 баллов

Полное: студент объясняет собственную модель и команды без механического чтения отчёта.

38. Шкала оценивания

Баллы	Оценка
90–100	отлично
70–89	хорошо
50–69	удовлетворительно
0–49	требуется повторное выполнение

39. Вопросы для защиты

1. Какую роль Redis выполняет в вашей информационной системе?
 2. Почему вы выбрали именно такое соглашение об именовании ключей?
 3. Почему для выбранного объекта используется Hash/String/List/Set?
 4. Какой TTL установлен для сессии и почему?
 5. Что произойдёт после окончания TTL?
 6. Как ваша система обрабатывает cache miss?
 7. Каким образом инвалидируется кэш после изменения основной БД?
 8. Какие данные в вашей модели наиболее чувствительны к устареванию?
 9. Какие проблемы возникнут при значительном росте числа пользователей?
 10. Какие данные вы принципиально не стали бы кэшировать на длительное время и почему?
-

40. Необязательное задание повышенной сложности

Исследование влияния стратегии TTL

Выберите один кэшированный объект и разработайте три стратегии:

TTL = 30 секунд

TTL = 5 минут

TTL = 1 час

Для каждой стратегии определите:

- предполагаемое количество cache hit;
- риск устаревания;
- частоту обращения к основной БД;
- подходящие сценарии.

Составьте таблицу:

TTL	Cache hit	Нагрузка на основную БД	Риск устаревания
30 с	ниже	выше	низкий
5 мин	средний	средняя	средний
1 час	выше	ниже	высокий для часто изменяемых данных

После анализа выберите TTL для своего сценария и обоснуйте решение.

За качественное выполнение преподаватель может учесть задание при выставлении баллов по разделам «Сложные операции» и «Анализ результатов», однако итоговая оценка не может превышать **100 баллов**.

41. Дополнительное исследовательское задание

Проверьте влияние перезаписи ключа на TTL.

Выполните:

```
SET lab5:ttl "first" EX 120  
TTL lab5:ttl
```

Затем:

```
SET lab5:ttl "second"  
TTL lab5:ttl
```

Зафиксируйте результат и объясните его.

После этого предложите способ обновления значения, при котором приложение осознанно управляет временем жизни кэша.

Сделайте архитектурный вывод:

управление TTL является частью проектирования кэша, а не дополнительной технической настройкой.

42. Итоговая схема лабораторной работы

В результате студент должен продемонстрировать полный цикл:

Требования информационной системы



Определение паттернов доступа



Выбор данных для Redis



Проектирование пространства ключей

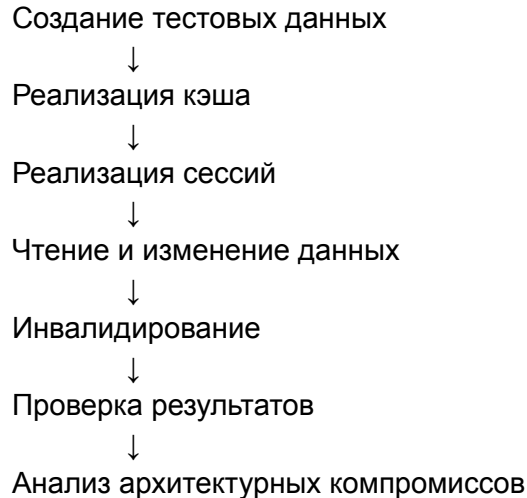


Выбор структур Redis



Определение TTL





Основной результат лабораторной работы — не набор выполненных команд Redis, а **обоснованная модель применения Redis в информационной системе**, в которой студент может объяснить назначение каждого ключа, структуры данных, TTL и стратегии инвалидирования.

43. Источники

1. **Силлабус дисциплины «Проектирование и применение NoSQL-технологий», 2026–2027 учебный год.** Тематический план: лекция № 5 и лабораторная работа № 5.
 2. **Redis Ltd. Redis Documentation.** Redis data types, commands and key expiration. Официальная документация Redis. Дата обращения: 27.09.2026.
 3. **Redis Ltd. SET command documentation.** Описание **SET**, параметров **EX**, **PX**, **NX**, **XX**, **KEEPTTL**. Дата обращения: 27.09.2026.
 4. **Redis Ltd. EXPIRE command documentation.** Управление временем жизни ключей. Дата обращения: 27.09.2026.
 5. **Redis Ltd. TTL command documentation.** Проверка оставшегося времени жизни ключа. Дата обращения: 27.09.2026.
 6. **Redis Ltd. Redis Hashes documentation.** Работа со структурой Hash и соответствующими командами. Дата обращения: 27.09.2026.
-

Краткая памятка студенту

Перед защитой убедитесь, что можете самостоятельно объяснить:

Почему Redis?

Почему этот ключ?

Почему этот тип данных?

Почему такой TTL?

Что произойдёт при cache miss?

Когда кэш станет устаревшим?

Как он инвалидируется?

Что произойдёт после завершения сессии?

Если студент может аргументированно ответить на эти вопросы и воспроизвести основные операции, цель лабораторной работы достигнута.